Session 4

Linear Classification, SVM

Given: Training Set

A dataset D with N labeled instance

Dataset: $D = \{(x(i), y(i))\}_{i=1}^{N}$

$y(i) \in \{1, ..., K\}$

Goal: Assign input x to one of K classes

	Number of times pregnant	Glucose	Blood Pressure	Skin Thickness	Insulin	Diabetes pedigree function	Age	BMI	Label
Patient 1	6	148	72	35	0	0.627	50	33.6	Positive
Patient 2	1	85	66	29	0	0.351	31	26.6	Negative
Patient 3	0	137	40	35	168	2.288	33	43.1	Positive
Patient 4	1	89	66	23	94	0.167	21	28.1	Negative
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.	.

A Discriminant Function is a mathematical function used to classify an input vector by assigning a score to it. The function maps an input x to a real number $g(x)$, which represents the confidence of assigning x to a particular class.

For **binary classification**, two functions $g_1(x)$ and $g_2(x)$are used for classes C_1 and C_2, respectively. The decision rule is:

$$\hat{y} = \begin{cases} C_1, \ g_1(x) \ > \ g_2(x) \\ C_2, \ otherwise \end{cases}$$

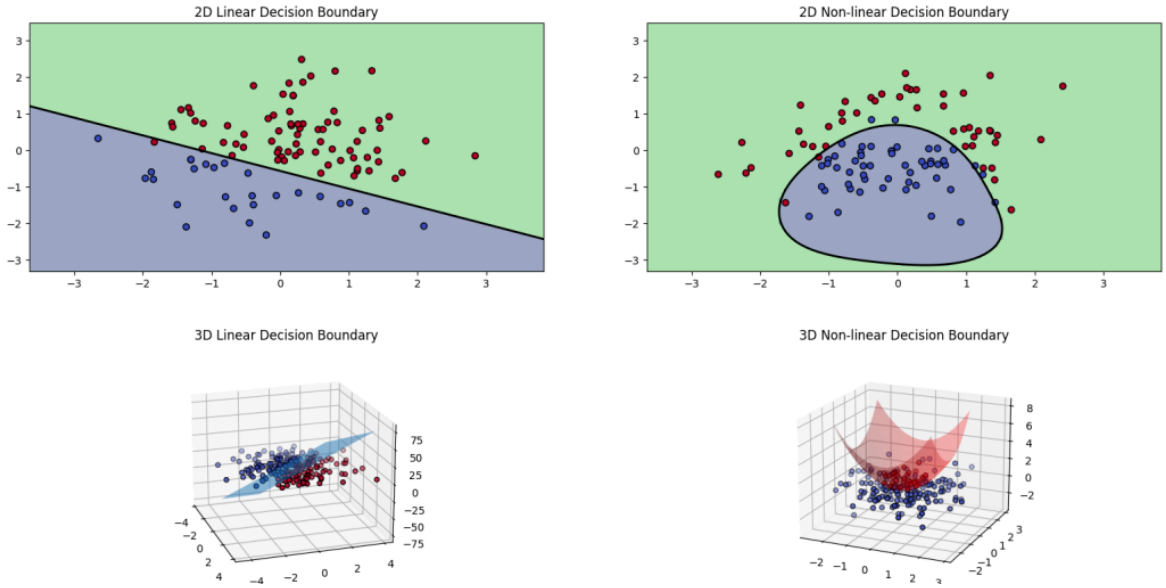
Multi-Class Classification:

For k-class problems, we compute gi(x) for every class i, and assign x to class with highest score:

$\hat{y} = argmax_i g_i(x)$

Decision Boundary

A dividing hyperplane that separates different classes in a feature space, also known as "Decision Surface".



Function: For two-category problem, we can only find a function $g : R^d \rightarrow R$

- $g_1(x) = g(x)$,
- $g_2(x) = -g(x)$

Decision Boundary: $g(x) = 0$

3. Linear Classifiers

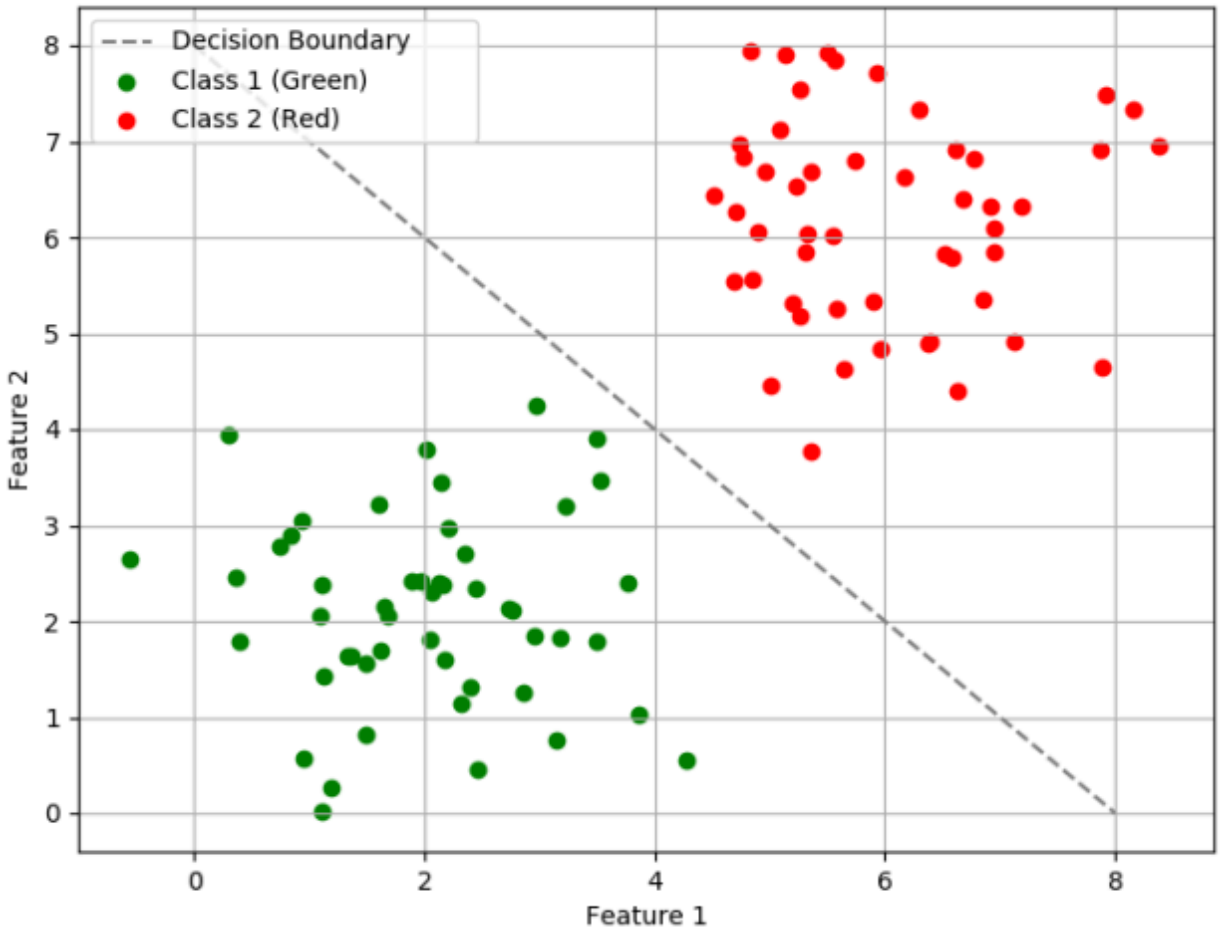
Characteristics

In case of linear classifiers, decision boundaries are linear in$d(x \in R^d)$, or linear in some given set of functions of x.

Linearly separable data: Data points that can be exactly separated by a linear decision boundary.

- Simplicity, Efficiency, Effectiveness

Mathematical Representation



- $g(x) = w^T x + w_0$
- $x = [x_1 ... x_d]$

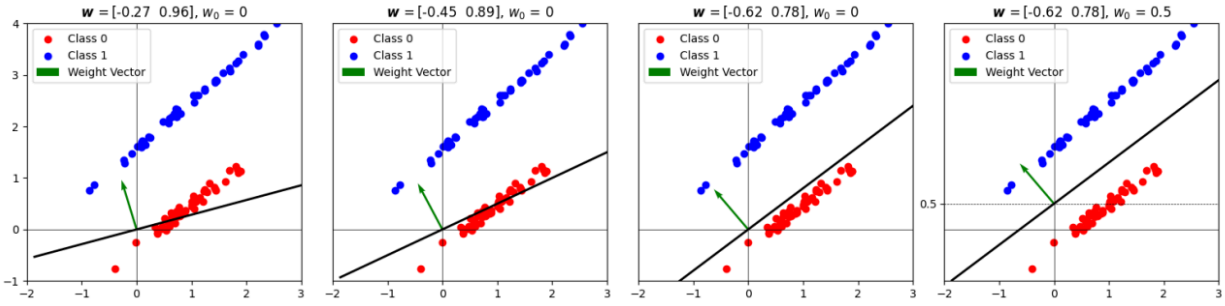
- $w = [w_1 ... w_d]$
- w_0: bias term

$$\hat{y} = \begin{cases} C_1, \ if \ w^T x \ + \ w_0 \ \geq \ 0 \\ C_2, \ otherwise \end{cases}$$

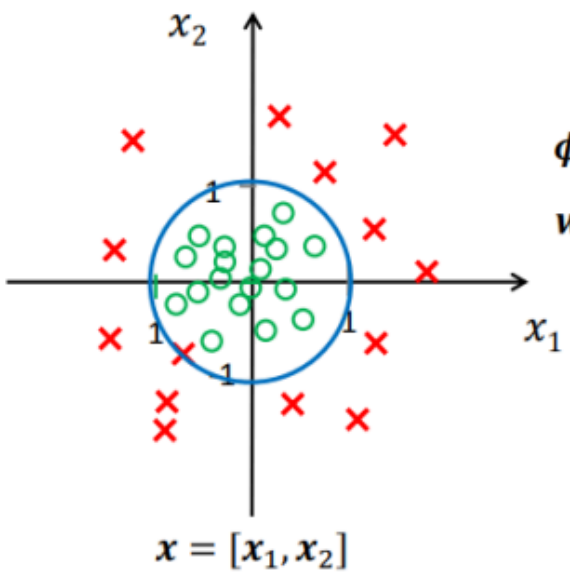
Decision Boundary is a $(d-1)$-dimensional hyperplane H in the $d-$dimensional feature space. Some properties of H are:

Orientation of H is determined by the normal vector $\left[\frac{w}{\|w_1\|}, ..., \frac{w}{\|w_d\|}\right]$.

w_0 determines the location of the surface.



Non-Linear Decision Boundaries



$$-1 + x_1^2 + x_2^2 = 0$$

$$\phi(x) = [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]$$

$$w = [w_0, w_1, \dots, w_m] = [-1, 0, 0, 1, 1, 0]$$

if $w^T \phi(x) \geq 0$ then $y = 1$
else $y = -1$

Achieved through feature transformation:

- Linear in transformed space
- Non-linear in original space

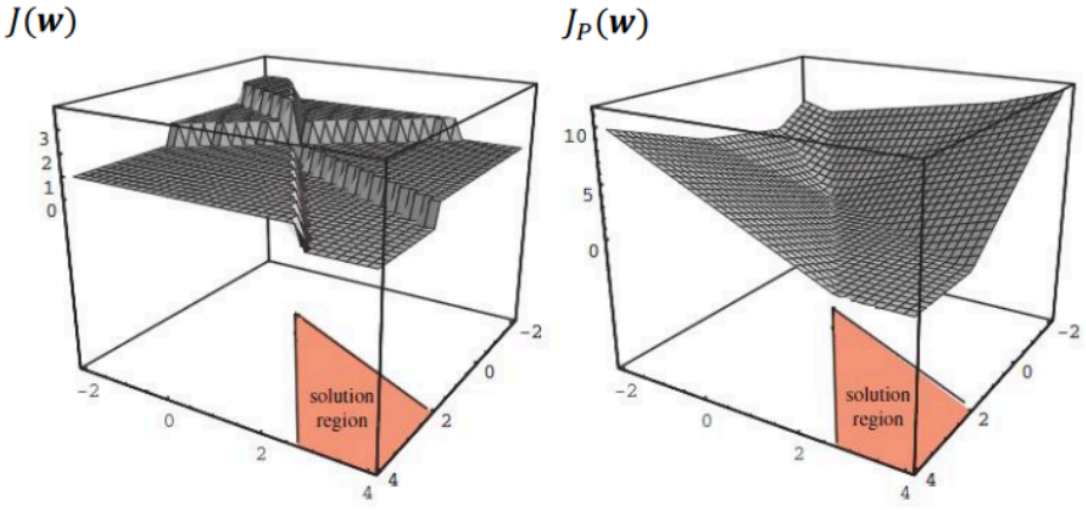
4. Perceptron

5. Cost Functions

Approaches

- **Sum of Squared Error (SSE)**
  - $J(w) = \Sigma(y(i) - \hat{y}(i))^2$
  - Better for regression
  - Prone to overfitting
- **Misclassification Count**

- Directly measures incorrect predictions
- Challenges with optimization



Focuses on misclassified points

$Jp(w) = -\Sigma y(i) w^T x(i)$

SVM

Vladimir Vapnik is credited with developing support vector machines (SVMs).

He was born in the **Soviet Union (now Russia)** and later moved to the **United States**.

Conducted most of his research at **AT&T Bell Labs**.

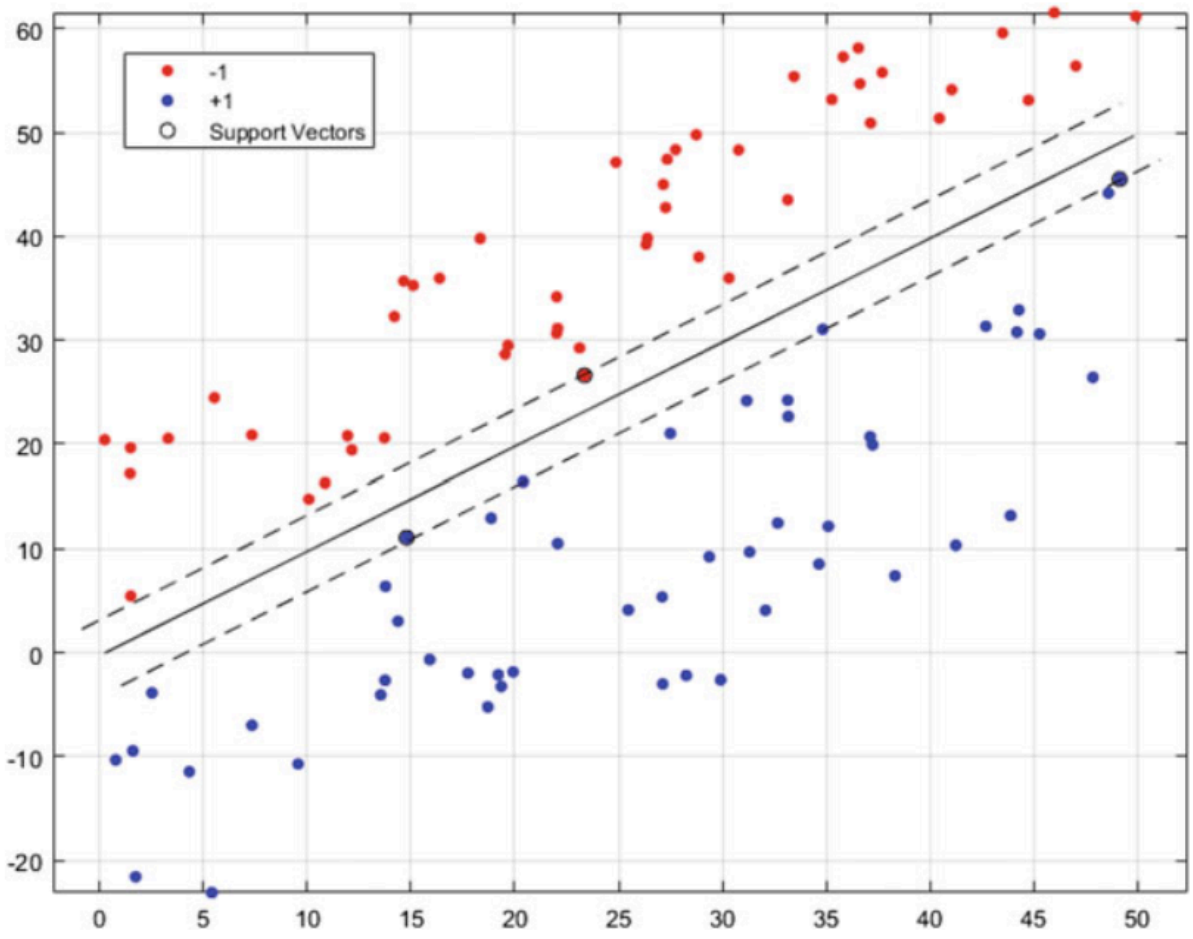
Co-developed **statistical learning theory** with **Chevronenkis**.

Focused on **optimal pattern recognition** using statistical methods in the 1960s.

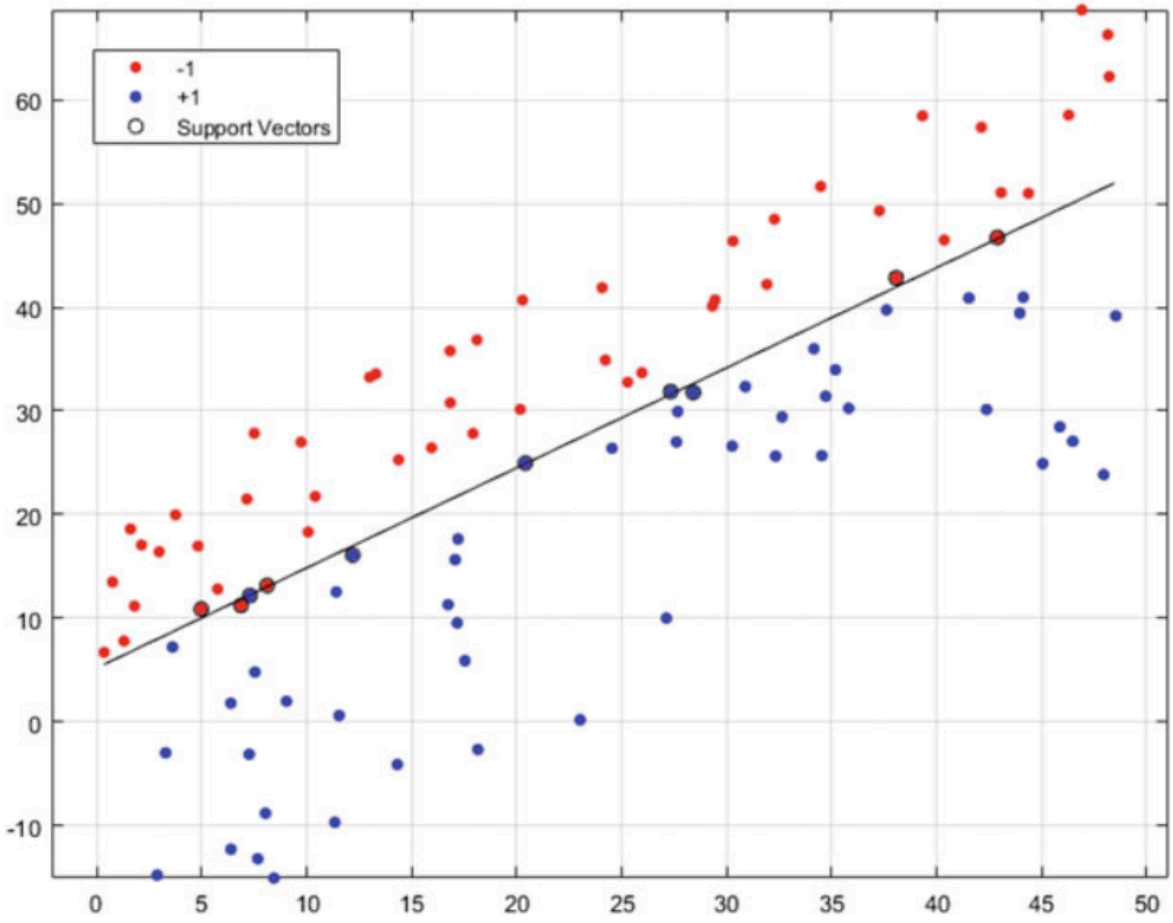
Introduced the concept of SVMs through the **generalized portrait algorithm** with **Alexander Lerner**.

Early applications of SVMs involved **classification** using an **optimal hyperplane** to maximize class separation.

- **SVM Algorithm**: Separates two classes with maximum margin using a subset of data points called **support vectors**.
- **Support Vectors**: Crucial data points that define class boundaries and determine the classification hyperplane.
- **Linearly Separable Case** : A simple scenario where a solid hyperplane optimally separates the classes, with dotted lines marking class boundaries.



- **Non-linearly Separable Case** : The algorithm still finds optimal support vectors and constructs a classification hyperplane.



- **Efficiency**: Once support vectors are identified, the rest of the dataset is unnecessary for classification, reducing storage and computation.
- **Comparison with k-NN**: SVM requires significantly less data for classification compared to algorithms like **k-nearest neighbors**.
- **Regularization**: The process of computing the hyperplane enhances **generalizability**, making SVM more robust.
- **Hyperplane Definition**: A hyperplane is a general term for a **linear decision boundary** extending into higher dimensions (dot in 1D, line in 2D, plane in 3D, and beyond).

Support Vector Machines (SVM) - Mathematical Theory

1. Theory of SVM

To understand how Support Vector Machines (SVMs) are trained, it is essential to grasp the underlying mathematical principles. The primary goal of SVM is to find a hyperplane that best separates two classes in an n-dimensional space.

1.1 Definition of the Hyperplane

Consider a binary classification problem with an n-dimensional dataset consisting of p pairs (x_i, y_i), where:

$$x_i \in \mathbb{R}^n, \quad y_i \in \{-1, +1\}$$

The equation of the hyperplane that separates the two classes with maximum margin is:

$$(w \cdot x) - w_0 = 0$$

- $w \in \mathbb{R}^n$ is the weight vector.
- w_0 is the bias term.

1.2 Constraint Formulation

For each training sample, the following constraints must hold:

$$y_i \left[(w \cdot x_i) - w_0\right] \geq 1, \quad \forall i = 1, \ldots, p$$

This ensures that the samples are correctly classified with a margin of at least 1.

1.3 Optimization Problem

To find the optimal hyperplane, we minimize the following regularization function:

$$\phi(w) = \tfrac{1}{2}||w||^2$$

subject to:

$$y_i \left[(w \cdot x_i) - w_0\right] \geq 1, \quad \forall i$$

This optimization problem is solved using **Lagrange multipliers**, leading to the dual formulation.

2. Lagrangian Formulation

Using Lagrangian multipliers $\alpha_i \geq 0$, the optimization problem becomes:

$$L(w, w_0, \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{p} \alpha_i \left[y_i \left((w \cdot x_i) - w_0\right) - 1\right]$$

where:

- L is the **Lagrangian function**.
- α are the **Lagrange multipliers**.

Applying **Karush-Kuhn-Tucker (KKT) conditions**, we obtain the solution:

$$\hat{w} = \sum_{\text{support vectors only}} y_i \alpha_i x_i$$

To handle non-linearly separable data, we introduce slack variables ξ_i to allow misclassified points:

$$y_i \left[(w \cdot x_i) - w_0\right] \geq 1 - \xi_i, \quad \xi_i \geq 0$$

The objective function is then modified as:

$$\min_w \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{p} \xi_i$$

where:

C is the **regularization parameter** that controls the tradeoff between margin size and misclassification.

4. Kernel Trick and Dual Form

For non-linearly separable data, **kernel functions** $K(x_i, x_j)$ transform data into a higher-dimensional space where it becomes linearly separable.

The dual form of the optimization problem is:

$$\max_\alpha \sum_{i=1}^{p} \alpha_i - \frac{1}{2} \sum_{i=1}^{p} \sum_{j=1}^{p} y_i y_j \alpha_i \alpha_j K(x_i, x_j)$$

subject to:

$$0 \leq \alpha_i \leq C, \quad \sum_{i=1}^{p} \alpha_i y_i = 0$$

Common kernels:

Linear Kernel:

$$K(x, x') = x \cdot x'$$

Polynomial Kernel:

$$K(x, x') = (x \cdot x' + c)^d$$

Radial Basis Function (RBF) Kernel:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

5. Generalization Error Decomposition

Generalization error measures model performance on unseen data:

$$E[(y - h_w(x))^2] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

where:

- **Bias**: Error due to incorrect model assumptions.
- **Variance**: Sensitivity to small data changes.
- **Noise**: Irreducible randomness in data.

Bias-Variance Tradeoff

- **High Bias, Low Variance** → Underfitting
- **Low Bias, High Variance** → Overfitting
- **Balanced Bias-Variance** → Optimal generalization

6. Model Assessment

Accuracy is one of the simplest and most commonly used performance metrics. It is defined as the ratio of correctly predicted instances to the total instances:

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalSamples}$$

	Predicted Negative	Predicted Positive
Actual Negative	990 (TN)	0 (FP)
Actual Positive	10 (FN)	0 (TP)

$$\text{Accuracy} = \frac{990 + 0}{1000} = 99\%$$

However, accuracy alone can be misleading, especially with imbalanced datasets.

Scenario: An alarm system can either ring or not ring when a thief is present.

Let's define the outcomes:

True Positive (TP): Alarm rings (correctly) when a thief is present.

True Negative (TN): Alarm does not ring (correctly) when no thief is present.

False Positive (FP): Alarm rings (incorrectly) when no thief is present (a false alarm).

False Negative (FN): Alarm does not ring (incorrectly) when a thief is present (a missed alarm)

	Thief Present	No Thief Present
Alarm Rings	TP	FP
Alarm Does Not Ring	FN	TN

Sensitivity (Recall):

$Sensitivity = \frac{TP}{TP+FN}$

Indicates the ability of the alarm system to correctly identify a thief. It is the proportion of actual positives (thief present) that are correctly identified.

Specificity:

$$Specificity = \frac{TN}{TN+FP}$$

Measures the ability of the alarm system to correctly identify when no thief is present. It is the proportion of actual negatives that are correctly identified.

Precision:

$$Precision = \frac{TP}{TP+FP}$$

Indicates the accuracy of the alarm when it rings. It is the proportion of times the alarm rang and a thief was indeed present out of all the times the alarm was activated.